

Ex.No : 5

Cache Mapping Simulation

Aim

To simulate **Direct-Mapped Cache Mapping** and **Fully Associative Cache Mapping** using **C programming** and observe the **cache hits** and **cache misses** for a given sequence of memory block references.

Algorithm

A. Direct-Mapped Cache Mapping

1. Initialize all cache lines as empty.
2. Define the cache size and the sequence of memory block references.
3. For each memory block:
 - o Calculate the cache line using:

$$\text{Cache Line} = \text{Block Number} \bmod \text{Cache Size}$$

- o If the memory block is already present in the calculated cache line, count it as a **Cache Hit**.
 - o Otherwise, count it as a **Cache Miss** and replace the block in that cache line.
4. Display the total number of **cache hits** and **cache misses**.

B. Fully Associative Cache Mapping (FIFO Replacement)

1. Initialize all cache lines as empty.
2. Define the cache size and the sequence of memory block references.
3. For each memory block:
 - o Search all cache lines.
 - o If the block is found, count it as a **Cache Hit**.
 - o Otherwise, count it as a **Cache Miss**.
 - o If the cache has empty locations, place the block in the next available cache line.
 - o If the cache is full, replace the oldest block using the **FIFO (First-In-First-Out)** replacement policy.
4. Display the total number of **cache hits** and **cache misses**.

Program (C)

```
#include <stdio.h>
```

```
int main()
{
    int cacheSize = 4;
    int blocks[] = {1, 2, 3, 4, 1, 5, 2, 1};
    int n = 8;

    int cache[10];
    int i, j, hit = 0, miss = 0;
```

```

printf("DIRECT MAPPED CACHE\n");

for(i = 0; i < cacheSize; i++)
    cache[i] = -1;

for(i = 0; i < n; i++)
{
    int line = blocks[i] % cacheSize;

    if(cache[line] == blocks[i])
    {
        printf("Block %d -> Hit\n", blocks[i]);
        hit++;
    }
    else
    {
        printf("Block %d -> Miss\n", blocks[i]);
        cache[line] = blocks[i];
        miss++;
    }
}

printf("\nHits = %d", hit);
printf("\nMisses = %d\n", miss);

printf("\nFULLY ASSOCIATIVE CACHE (FIFO)\n");

for(i = 0; i < cacheSize; i++)
    cache[i] = -1;

hit = miss = 0;

    int front = 0, count = 0;

for(i = 0; i < n; i++)
{
    int found = 0;

    for(j = 0; j < cacheSize; j++)
    {
        if(cache[j] == blocks[i])
        {
            found = 1;
            break;
        }
    }

    if(found)
    {
        printf("Block %d -> Hit\n", blocks[i]);
    }
}

```

```

        hit++;
    }
    else
    {
        printf("Block %d -> Miss\n", blocks[i]);
        miss++;

        if(count < cacheSize)
        {
            cache[count++] = blocks[i];
        }
        else
        {
            cache[front] = blocks[i];
            front = (front + 1) % cacheSize;
        }
    }
}

printf("\nHits  = %d", hit);
printf("\nMisses = %d", miss);

return 0;
}

```

Output

DIRECT MAPPED CACHE

Block 1 -> Miss
 Block 2 -> Miss
 Block 3 -> Miss
 Block 4 -> Miss
 Block 1 -> Hit
 Block 5 -> Miss
 Block 2 -> Hit
 Block 1 -> Hit

Hits = 3
 Misses = 5

FULLY ASSOCIATIVE CACHE (FIFO)

Block 1 -> Miss
 Block 2 -> Miss
 Block 3 -> Miss
 Block 4 -> Miss
 Block 1 -> Hit
 Block 5 -> Miss

Block 2 -> Hit
Block 1 -> Miss

Hits = 2
Misses = 6

Observation

Cache Mapping Technique	Hits	Misses
Direct-Mapped Cache	3	5
Fully Associative Cache (FIFO)	2	6

Result

Thus, the **Direct-Mapped Cache Mapping** and **Fully Associative Cache Mapping** techniques were successfully simulated using **C programming**. The cache hits and cache misses for the given sequence of memory block references were obtained and compared. The simulation demonstrates the behavior and performance of different cache mapping techniques for memory access operations.